src\case_studies\E_blockbeam\pid_controller.py

Generated: 2026-02-26 12:33:37

pid_controller.py
src\case_studies\E_blockbeam\pid_controller.py

```python
# 3rd-party
import numpy as np

# local (controlbook)
from . import params as P
from .. import common
from ..control.pd import PD
from ..control.pid import PID
from ..control.dirty_derivative_filter import DirtyDerivativeFilter
from ..control import utils_design


class BlockbeamControllerPID(common.ControllerBase):
    def __init__(self):
        # tuning parameters
        tr_theta = 0.12
        zeta_theta = 0.707
        M = 10  # time separation factor between inner and outer loop
        tr_z = tr_theta * M
        zeta_z = 0.707
        ki_z = -0.1

        # system parameters
        a1_inner, a0_inner = P.tf_inner_den[-2:]
        a1_outer, a0_outer = P.tf_outer_den[-2:]
        b0_inner = P.tf_inner_num[-1]
        b0_outer = P.tf_outer_num[-1]

        # Inner loop
        alpha1_inner, alpha0_inner = utils_design.get_des_CE(tr_theta, zeta_theta)
        kp_theta = (alpha0_inner - a0_inner) / b0_inner
        kd_theta = (alpha1_inner - a1_inner) / b0_inner
        self.theta_pd = PD(kp_theta, kd_theta)
        print(f"Inner loop: {kp_theta = :.3f}, {kd_theta = :.3f}")

        # DC gain of inner loop
        DC_gain = kp_theta * b0_inner / (a0_inner + kp_theta * b0_inner)
        print(f"{DC_gain = }")  # a0 is 0, so DC_gain = 1

        # Outer loop
        alpha1_outer, alpha0_outer = utils_design.get_des_CE(tr_z, zeta_z)
        effective_b0_outer = b0_outer * DC_gain
        kp_z = (alpha0_outer - a0_outer) / effective_b0_outer
        kd_z = (alpha1_outer - a1_outer) / effective_b0_outer
        self.z_pid = PID(kp_z, ki_z, kd_z, P.ts, anti_windup_vel_tol=0.1)
        print(f"Outer loop: {kp_z = :.3f}, {ki_z = :.3f}, {kd_z = :.3f}")

        # dirty derivative filters (derivatives calculated outside PD/PID classes)
        self.z_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05, x0=P.z0)
        self.theta_dot_filter = DirtyDerivativeFilter(P.ts, sigma=0.05, x0=P.theta0)

    def update_with_state(self, r, x):
        raise NotImplementedError("This controller only uses measurement feedback.")

    def update_with_measurement(self, r, y):
        z, theta = y
        z_ref = r[0]

        # calculate derivatives externally using dirty derivative filters
        z_dot = self.z_dot_filter.update(z)
        theta_dot = self.theta_dot_filter.update(theta)
```

```
        # outer loop control with explicit derivative
        theta_ref = self.z_pid.update_modified(z_ref, z, z_dot)
        theta_ref = min(max(np.radians(-10), theta_ref), np.radians(10))
        r[1] = theta_ref  # if you want to visualize the "reference" angle

        # inner loop control with explicit derivative
        F_tilde = self.theta_pd.update_modified(theta_ref, theta, theta_dot)
        F_fl = P.m1 * P.g * (z / P.length) + P.m2 * P.g / 2.0
        F_unsat = F_tilde + F_fl
        u_unsat = np.array([F_unsat])
        u = self.saturate(u_unsat, u_max=P.force_max)

        # anti-windup
        # self.z_pid.add_anti_windup_saturation(u, u_unsat)

        # include the following if you want to visualize dirty derivatives
        xhat = np.array([z, z_dot, theta, theta_dot])

        return u, xhat.astype(np.float64)
```